TransitOps — Smart Transport Operations Platform

Complete Build Specification (for AI-assisted development)

Build a full-stack transport operations platform. Follow this spec exactly. Do not add features beyond what is listed. Prioritize correctness of business rules over visual polish.

---

1. Tech Stack (fixed — do not substitute)

- **Backend:** Python, FastAPI, SQLAlchemy (sync, declarative), PostgreSQL, Pydantic v2
- **Package manager:** uv (`uv add ...`, `uv run ...`) — no pip, no requirements.txt
- **Auth:** JWT (python-jose), passlib[bcrypt] for password hashing
- **Frontend:** React 18 + Vite, **JavaScript only (NO TypeScript)**, Tailwind CSS, react-router-dom, axios, recharts
- **No Redux.** Auth state via React Context + useReducer. Everything else uses local useState + fetch-on-mount.
- DB tables created with `Base.metadata.create_all()` on startup. No Alembic.
- Status fields stored as **String** columns (validated by Pydantic), not Postgres ENUMs.

2. Project Structure

```
transitops/
  backend/
    app/
      main.py            # FastAPI app, CORS, include routers, create_all, seed call
      database.py        # engine, SessionLocal, Base, get_db dependency
      models.py          # ALL SQLAlchemy models in one file
      schemas.py         # ALL Pydantic schemas in one file
      auth.py            # hash/verify password, create/decode JWT, get_current_user
      deps.py            # require_role(*roles) dependency factory
```

```
    seed.py              # idempotent seed data (skip if users exist)
    routers/
      auth.py            # POST /auth/signup, POST /auth/login, GET /auth/me
      vehicles.py
      drivers.py
      trips.py
      maintenance.py
      fuel.py
      expenses.py
      dashboard.py
      reports.py
  pyproject.toml         # managed by uv
  .env                   # DATABASE_URL, JWT_SECRET
frontend/
  src/
    api/client.js        # axios instance, baseURL, JWT interceptor from localStorage
    context/AuthContext.jsx
    components/
      layout/Sidebar.jsx, Navbar.jsx, ProtectedRoute.jsx
      shared/StatusBadge.jsx, DataTable.jsx, KpiCard.jsx, Modal.jsx
    pages/
```

Login.jsx, Signup.jsx, Dashboard.jsx, Vehicles.jsx, Drivers.jsx,

Trips.jsx, Maintenance.jsx, FuelExpenses.jsx, Reports.jsx

App.jsx

main.jsx

3. Database Models (SQLAlchemy)

users

column	**type**	**notes**
id	Integer PK	
name	String	required
email	String	unique, indexed
password_hash	String	
role	String	one of: `fleet_manager`, `driver`, `safety_officer`, `financial_analyst`
status	String	`active` / `inactive`, default `active`
created_at	DateTime	default now

vehicles

column	**type**	**notes**
id	Integer PK	
reg_number	String	**unique**, indexed
name	String	vehicle name/model
type	String	e.g. van, truck, bike
max_load_capacity	Float	kg

column	type	notes
odometer	Float	km
acquisition_cost	Float	
status	String	`available` / `on_trip` / `in_shop` / `retired`, default `available`
region	String	nullable
created_at	DateTime	

drivers

column	type	notes
id	Integer PK	
name	String	
license_number	String	unique
license_category	String	
license_expiry	Date	
contact_number	String	
safety_score	Float	default 100
status	String	`available` / `on_trip` / `off_duty` / `suspended`, default `available`
created_at	DateTime	

trips

column	type	notes
id	Integer PK	
source	String	

column	type	notes
destination	String	
vehicle_id	FK → vehicles.id	
driver_id	FK → drivers.id	
cargo_weight	Float	
planned_distance	Float	km
actual_distance	Float	nullable, set on completion
fuel_consumed	Float	nullable, set on completion (liters)
fuel_efficiency	Float	nullable, computed = actual_distance / fuel_consumed on completion
status	String	`draft` / `dispatched` / `completed` / `cancelled`, default `draft`
created_at, dispatched_at, completed_at	DateTime	nullable except created_at

maintenance_logs

column	type	notes
id	Integer PK	
vehicle_id	FK	
issue_description	String	
cost	Float	default 0
status	String	`active` / `closed`, default `active`
created_at, closed_at	DateTime	closed_at nullable

fuel_logs

| id PK | vehicle_id FK | liters Float | cost Float | date Date |

expenses

| id PK | vehicle_id FK | type String (`toll`/`other`) | amount Float | date Date |

4. Business Rules (CRITICAL — implement exactly, all server-side)

All rules enforced in the **backend**, inside a single DB transaction per action. Frontend must never be the only place a rule lives.

4.1 Trip dispatch — `POST /trips/{id}/dispatch`

Reject with HTTP 400 + clear error message if ANY of:

1. Trip status is not `draft`
2. Vehicle status is not `available` (covers on_trip, in_shop, retired)
3. Driver status is not `available` (covers on_trip, off_duty, suspended)
4. Driver `license_expiry` < today
5. `cargo_weight` > vehicle `max_load_capacity`

On success (single transaction):

- trip.status = `dispatched`, trip.dispatched_at = now
- vehicle.status = `on_trip`
- driver.status = `on_trip`

4.2 Trip completion — `POST /trips/{id}/complete`

Body: `{ actual_distance, fuel_consumed, final_odometer }`

- Only allowed if trip.status == `dispatched`
- Sets trip.status = `completed`, completed_at = now
- Computes trip.fuel_efficiency = actual_distance / fuel_consumed (guard divide-by-zero → null)
- Updates vehicle.odometer = final_odometer (must be >= current odometer, else 400)
- Auto-creates a fuel_log row for the vehicle? **No** — fuel logs are manual entries; do not auto-create.
- vehicle.status = `available`, driver.status = `available`

4.3 Trip cancel — `POST /trips/{id}/cancel`

- Allowed if status is `draft` or `dispatched`
- If it was `dispatched`: restore vehicle.status = `available`, driver.status = `available`
- trip.status = `cancelled`

4.4 Maintenance — `POST /maintenance`

- Creating an `active` maintenance record sets vehicle.status = `in_shop` immediately
- Reject (400) if vehicle is currently `on_trip` (cannot service a vehicle mid-trip)

4.5 Maintenance close — `POST /maintenance/{id}/close`

- Sets status = `closed`, closed_at = now
- Restores vehicle.status = `available` **unless** vehicle.status == `retired`

4.6 Registry rules

- Vehicle reg_number unique → return 400 with friendly message on duplicate
- Driver license_number unique → same
- Retired or in_shop vehicles must be excluded from the vehicle dropdown when creating trips (backend provides `GET /vehicles?status=available` filter; frontend uses it)
- Suspended/off-duty/expired-license drivers excluded from driver dropdown the same way (`GET /drivers?assignable=true` returns only available drivers with unexpired licenses)

5. API Endpoints

Auth

- `POST /auth/signup` — { name, email, password, role } → creates user. (Hackathon simplification: role selectable at signup.)
- `POST /auth/login` — { email, password } → { access_token, user }
- `GET /auth/me` — current user from JWT

RBAC (enforced via require_role dependency)

- Vehicles CRUD, maintenance approve/close: `fleet_manager`
- Trips create/dispatch/complete/cancel: `fleet_manager`, `driver`
- Drivers CRUD, driver suspend: `fleet_manager`, `safety_officer`
- Fuel logs / expenses create: `fleet_manager`, `financial_analyst`, `driver`
- Reports & dashboard: all authenticated roles (read-only)

Resources (all JWT-protected)

- `GET/POST /vehicles`, `GET/PATCH/DELETE /vehicles/{id}`, `GET /vehicles/{id}/history` (trips + maintenance + total cost)
- `GET/POST /drivers`, `GET/PATCH/DELETE /drivers/{id}`, `GET /drivers/expiring?days=30`
- `GET/POST /trips`, `GET /trips/{id}`, plus dispatch/complete/cancel actions above
- `GET/POST /maintenance`, `POST /maintenance/{id}/close`
- `GET/POST /fuel-logs`, `GET/POST /expenses`
- `GET /dashboard/kpis`
- `GET /reports/vehicle-costs`, `GET /reports/vehicle-costs/csv` (CSV download via StreamingResponse)

Dashboard KPIs — `GET /dashboard/kpis` returns:

```
{
  "active_vehicles": 0,      // status != retired
  "available_vehicles": 0,
  "in_maintenance": 0,       // status == in_shop
  "active_trips": 0,         // status == dispatched
  "pending_trips": 0,        // status == draft
  "drivers_on_duty": 0,      // status in (available, on_trip)
  "fleet_utilization": 0.0   // on_trip vehicles / active vehicles * 100
}
```

Reports data

- Per-vehicle: total fuel cost, total maintenance cost, total expenses, **operational cost = fuel + maintenance + expenses**, avg fuel efficiency (from completed trips), trip count
- CSV export of that table

6. Frontend Pages & Behavior

- **Login / Signup** — forms, store JWT in localStorage, redirect to /dashboard

- **Layout** — sidebar nav (Dashboard, Vehicles, Drivers, Trips, Maintenance, Fuel & Expenses, Reports), navbar with user name + role badge + logout + dark mode toggle
- **Dashboard** — 7 KPI cards from /dashboard/kpis; a recharts bar chart of operational cost per vehicle; a recharts pie of vehicle status distribution; **yellow warning banner listing drivers whose license expires within 30 days** (from /drivers/expiring)
- **Vehicles** — table (reg number, name, type, capacity, odometer, status badge, region) with client-side search box + status filter dropdown; Add/Edit modal; row click opens a slide-over drawer with vehicle history (trips, maintenance, total cost)
- **Drivers** — table with search + status filter; Add/Edit modal; expired licenses shown with red badge; Suspend/Reactivate button (safety_officer/fleet_manager only)
- **Trips** — create form (source, destination, vehicle dropdown [available only], driver dropdown [assignable only], cargo weight, planned distance); trips table with status badges; per-row action buttons: Dispatch (draft), Complete (dispatched → opens modal asking actual_distance / fuel_consumed / final_odometer), Cancel; show backend validation errors as red toast/alert text verbatim
- **Maintenance** — create form (vehicle, issue, cost), table of logs, Close button on active rows
- **Fuel & Expenses** — two simple forms + two tables (fuel logs, expenses), filterable by vehicle
- **Reports** — per-vehicle cost table (fuel, maintenance, expenses, total, efficiency), Download CSV button
- **RBAC in UI** — hide action buttons the current role can't use (backend still enforces regardless)
- **Dark mode** — Tailwind `dark:` classes, toggle in navbar, preference in localStorage

7. Seed Data (seed.py — run on startup if users table empty)

- 4 users, one per role, all password `password123`:
  - fleet@transitops.com (fleet_manager), driver@transitops.com (driver), safety@transitops.com (safety_officer), finance@transitops.com (financial_analyst)
- 6 vehicles: include **Van-05 with max_load_capacity 500** (spec's example), one retired, one in_shop, varied types/regions
- 5 drivers: include **Alex with a valid license**, one with license expired last month, one suspended, one expiring in 15 days (to light up the warning banner)
- 3 trips in mixed states, 2 maintenance logs (1 active on the in_shop vehicle), a handful of fuel logs and expenses so Reports/Dashboard aren't empty

8. Acceptance Test (must pass end-to-end — this is the spec's example workflow)

1. Register vehicle Van-05, capacity 500 kg → status Available

2. Register driver Alex, valid license
3. Create trip, cargo 450 kg, select Van-05 + Alex
4. Dispatch → succeeds (450 ≤ 500); Van-05 and Alex both become On Trip
5. Try creating + dispatching another trip with Van-05 → rejected (vehicle on trip)
6. Try dispatching a trip with 600 kg cargo on Van-05 (after completing #4) → rejected (over capacity)
7. Complete trip #4 with final odometer + fuel → both statuses back to Available, fuel efficiency computed
8. Create maintenance record for Van-05 → status In Shop, vehicle disappears from trip form dropdown
9. Close maintenance → Available again
10. Dashboard KPIs and Reports reflect all of the above
11. Dispatch attempt with the expired-license driver → rejected
12. Duplicate reg_number registration → rejected with friendly error

9. Build Order

1. Backend: database.py, models.py, create_all, seed.py
2. Backend: auth (signup/login/JWT/get_current_user/require_role)
3. Backend: vehicles + drivers routers (CRUD + filters)
4. Backend: trips router with ALL dispatch/complete/cancel rules ← highest priority logic
5. Backend: maintenance, fuel, expenses routers
6. Backend: dashboard KPIs + reports + CSV
7. Frontend: Vite setup, Tailwind, axios client, AuthContext, ProtectedRoute, Login page, layout
8. Frontend: Vehicles + Drivers pages
9. Frontend: Trips page with action buttons + error display
10. Frontend: Maintenance, Fuel & Expenses pages
11. Frontend: Dashboard (KPIs, charts, license banner), Reports + CSV download
12. Frontend: search/filters, vehicle history drawer, RBAC-gated buttons, dark mode
13. Run the full Acceptance Test (section 8); fix failures

10. Conventions

- CORS: allow http://localhost:5173
- All datetimes UTC; dates as ISO strings in JSON
- Error responses: `{ "detail": "human readable message" }` (FastAPI default) — frontend displays `detail` directly
- Keep components small; no premature abstraction; single models.py / schemas.py files are fine
- Commit after each build-order step